

The background of the slide features a wireframe illustration of two hands, one on the left and one on the right, reaching towards the center. In the center, a glowing sphere is surrounded by several smaller, fainter spheres, all connected by a network of thin, light blue lines, suggesting a complex system or a neural network. The overall aesthetic is futuristic and technological.

Bir Fikirden Gerçek Zamanlı Oyuna: Derin Öğrenme ile Taş-Kağıt-Makas Tanıma Serüveni

Projenin tüm adımları, karşılaşılan zorluklar ve öğrenilen dersler

Serüven Başlıyor: Projemizin Adımları



1

**Temeller: Veri Seti
ve Araçların
Seçimi**

Nereden başlıyoruz?



2

**Faz 1: İlk Model:
Sıfırdan Bir CNN
İnşası**

_İlk denememiz ve
karşılaştığımız 'aşırı
öğrenme' problemi._



3

**Faz 2 & 3:
Optimizasyon:
Erken Durdurma ve
Veri Artırma**

_Modelimizi nasıl daha
akıllı ve dayanıklı hale
getirdik?_



4

**Faz 4: Profesyonel
Yaklaşım: Transfer
Learning
(MobileNetV2)**

_En yüksek başarıma
giden yol._



5

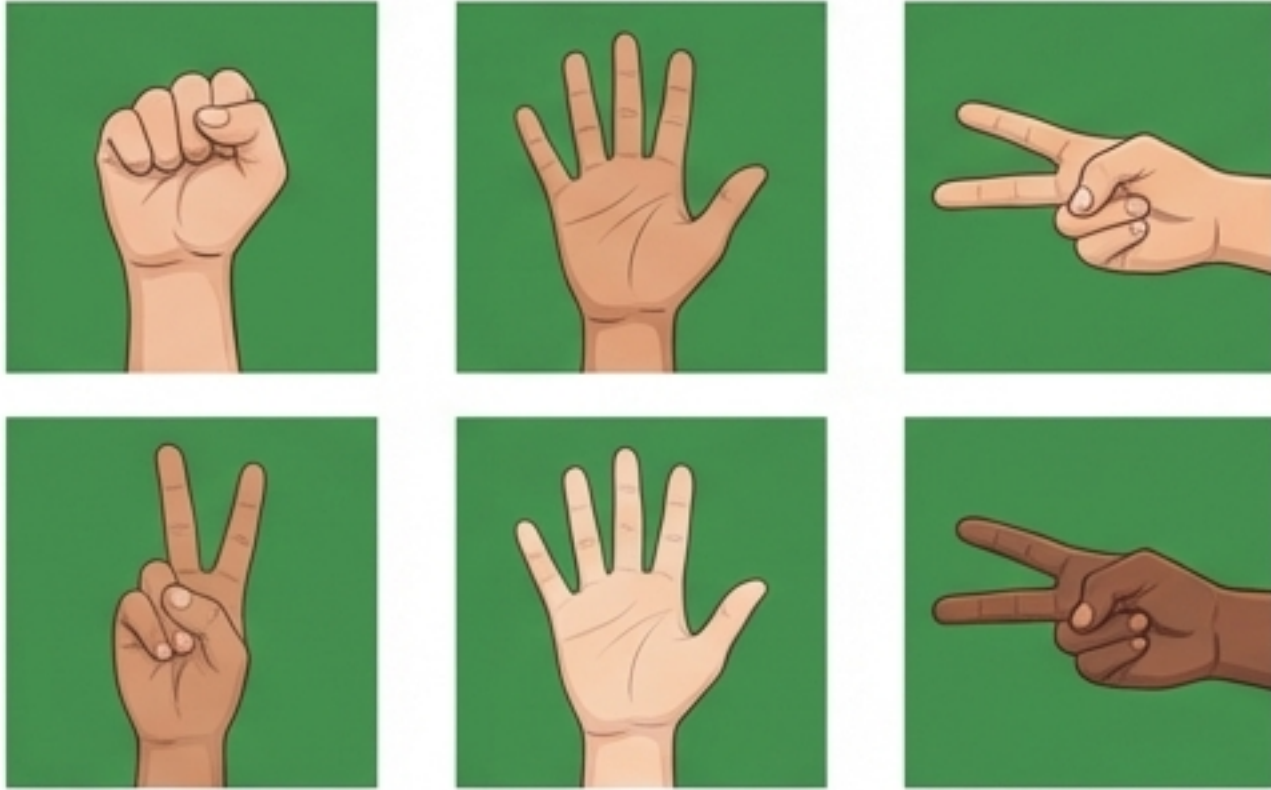
**Zirve: Gerçek
Zamanlı Oyun**

_Modelimizin OpenCV
ve MediaPipe ile hayata
geçışı._

Her Şeyin Başlangıcı: Veri Seti ve Araçlarımız

Veri Seti

- **Kaynak:** Kaggle - Rock, Paper, Scissors Images
- **İçerik:** 3 sınıfa ait (Taş, Kağıt, Makas) 1751 eğitim ve 437 doğrulama olmak üzere toplam ~2200 görüntü.
- **Ön İşleme:** Görüntüleri 150x150 piksele yeniden boyutlandırma. Piksel değerlerini 0-1 aralığına normalize etme (rescale=1./255).



Found 1751 images belonging to 3 classes.

Teknolojilerimiz



TensorFlow



Keras

Derin öğrenme modellerini inşa etmek, eğitmek ve doğrulamak için endüstri standardı.



Ücretsiz GPU erişimi ile hızlı model eğitimi için bulut tabanlı geliştirme ortamı.



Modelimizi gerçek zamanlı kamera görüntülerinde çalıştırmak...



MediaPipe

...ve elleri hassas bir şekilde tespit etmek için.

Cesur Bir Başlangıç: Kendi Modelimizi İnşa Ediyoruz

Mimari

Modelimiz, görüntüdeki özellikleri katman katman öğrenen klasik bir Evrişimli Sinir Ağı (CNN) yapısıdır.

1. **Conv2D & MaxPooling2D (x3):** Görüntüden 32, 64, ve 128 filtre ile sırasıyla temel (kenarlar), orta (şekiller) ve karmaşık (el pozisyonları) özellikleri çıkarır.
2. **Flatten:** Öğrenilen 2D özellikleri tek boyutlu bir vektöre dönüştürür.
3. **Dropout (0.5):** Ezberlemeyi (overfitting) önlemek için nöronların %50'sini rastgele kapatır.
4. **Dense (512 & 3):** Özellikleri birleştirerek son kararı verir (Taş, Kağıt veya Makas).

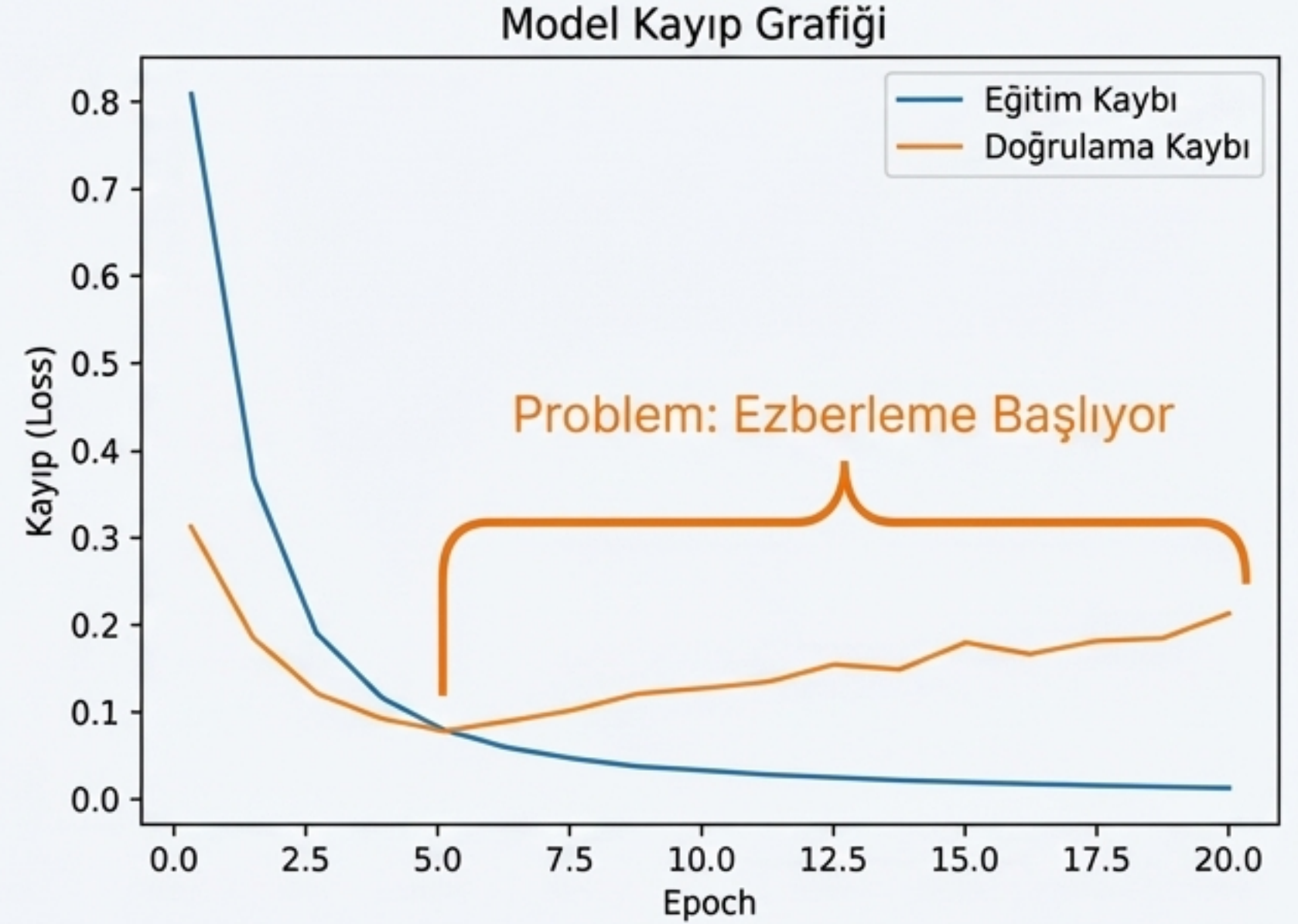
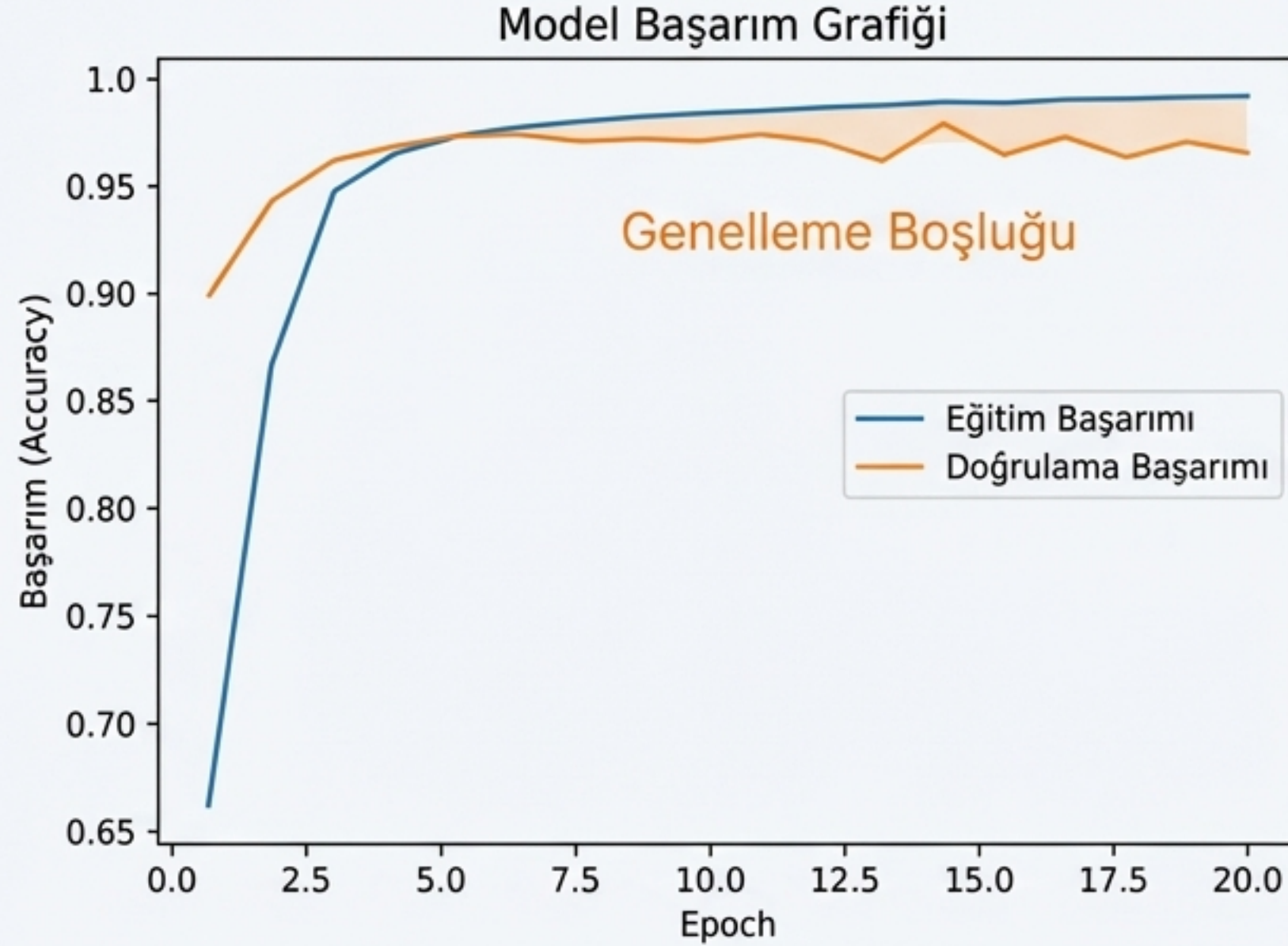
```
model = Sequential([
    # 1. EVRİŞİM BLOĞU
    Conv2D(32, (3,3), activation='relu',
           input_shape=(IMG_SIZE, IMG_SIZE, 3)),
    MaxPooling2D(2, 2),

    # 2. EVRİŞİM BLOĞU
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2, 2),

    # 3. EVRİŞİM BLOĞU
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2, 2),

    # KARAR VERME BLOĞU
    Flatten(),
    Dropout(0.5),
    Dense(512, activation='relu'),
    Dense(3, activation='softmax')
])
```

İlk Sonuçlar: Modelimiz Öğreniyor... Ama Fazlasıyla!



Teşhis: Aşırı Öğrenme (Overfitting)

Modelimiz eğitim verilerini 'ezberliyor' ancak daha önce görmediği yeni verilerde aynı başarıyı gösteremiyor. Genelleme yapamıyor.

Daha Akıllı Eğitim: Gereksiz Tekrarı Önlemek

Problem:

Model, en iyi genelleme performansına ulaştığı noktayı geçtikten sonra bile eğitime devam ederek doğrulama verilerindeki performansını düşürüyor.

Çözüm: `EarlyStopping` Callback'i

Modelin doğrulama kaybını (`val\_loss`) izlemesini ve belirli bir 'sabır' (`patience=5`) süresi boyunca iyileşme göstermezse eğitimi otomatik olarak durdurmasını sağlıyoruz.

`restore\_best\_weights=True` ile en iyi performansı gösterdiği andaki ağırlıkları geri yüklüyoruz.

```
early_stopping = EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True,  
    verbose=1  
)
```

Optimizasyonun Gücü: Tam Zamanında Durdurulan Eğitim

Ne Değişti?

- Eğitim, 20 epoch'un tamamını bitirmek yerine, 17. epoch'ta **en düşük doğrulama kaybına** ulaştıktan sonra durdu.

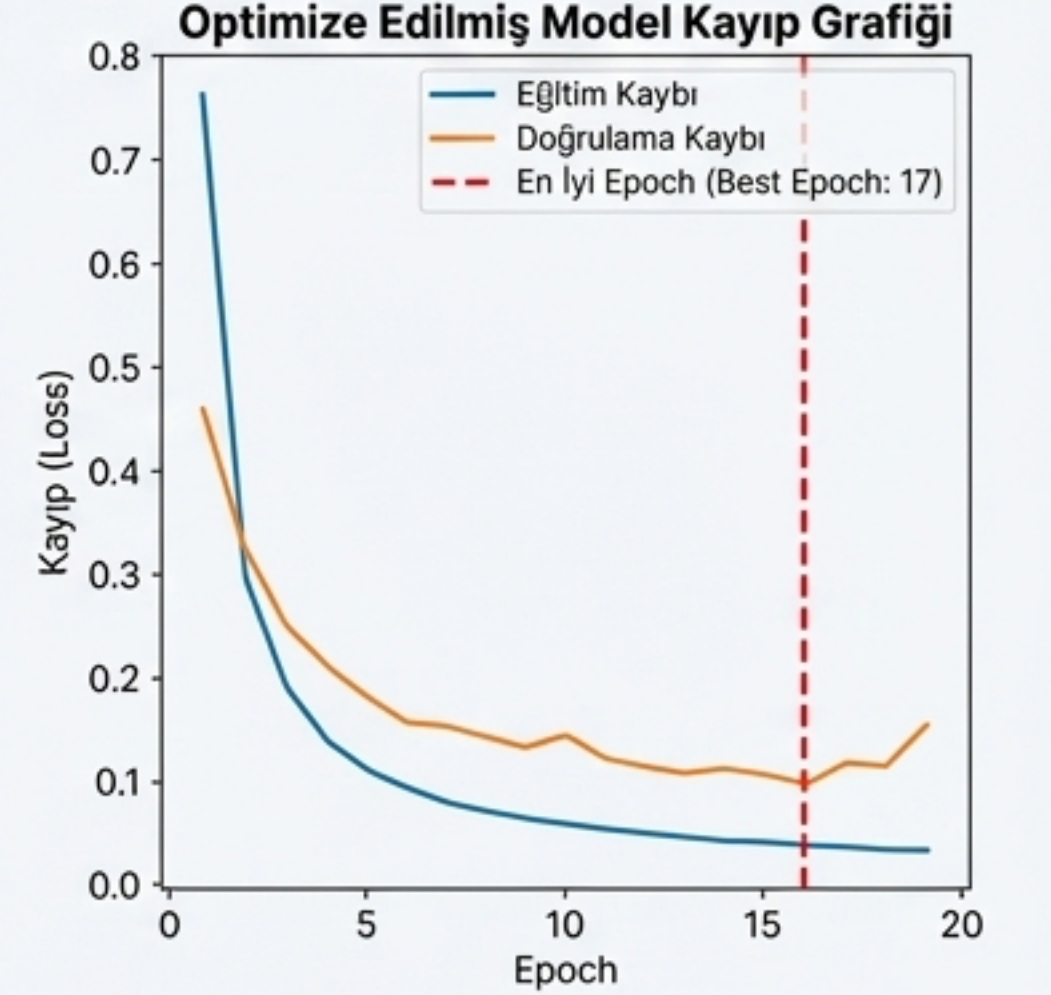
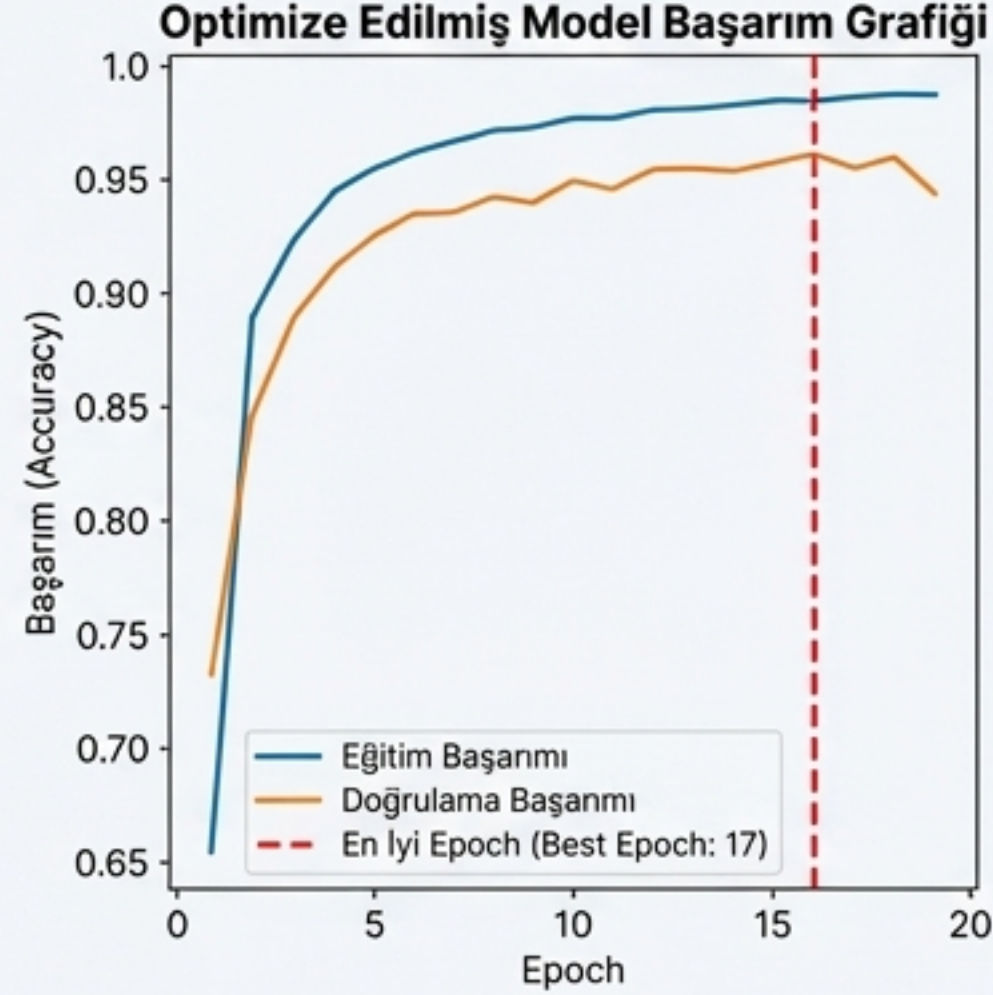


Restoring model weights from the end of the best epoch: 17.

- Grafikteki kırmızı çizgi, modelin en iyi genelleme yeteneğine sahip olduğu 'altın anı' gösteriyor.

Kazanım:

Hem gereksiz eğitimden kaynaklanan zaman kaybını önledik hem de modelin ezberlemesinin önüne geçerek daha güvenilir bir sonuç elde ettik.



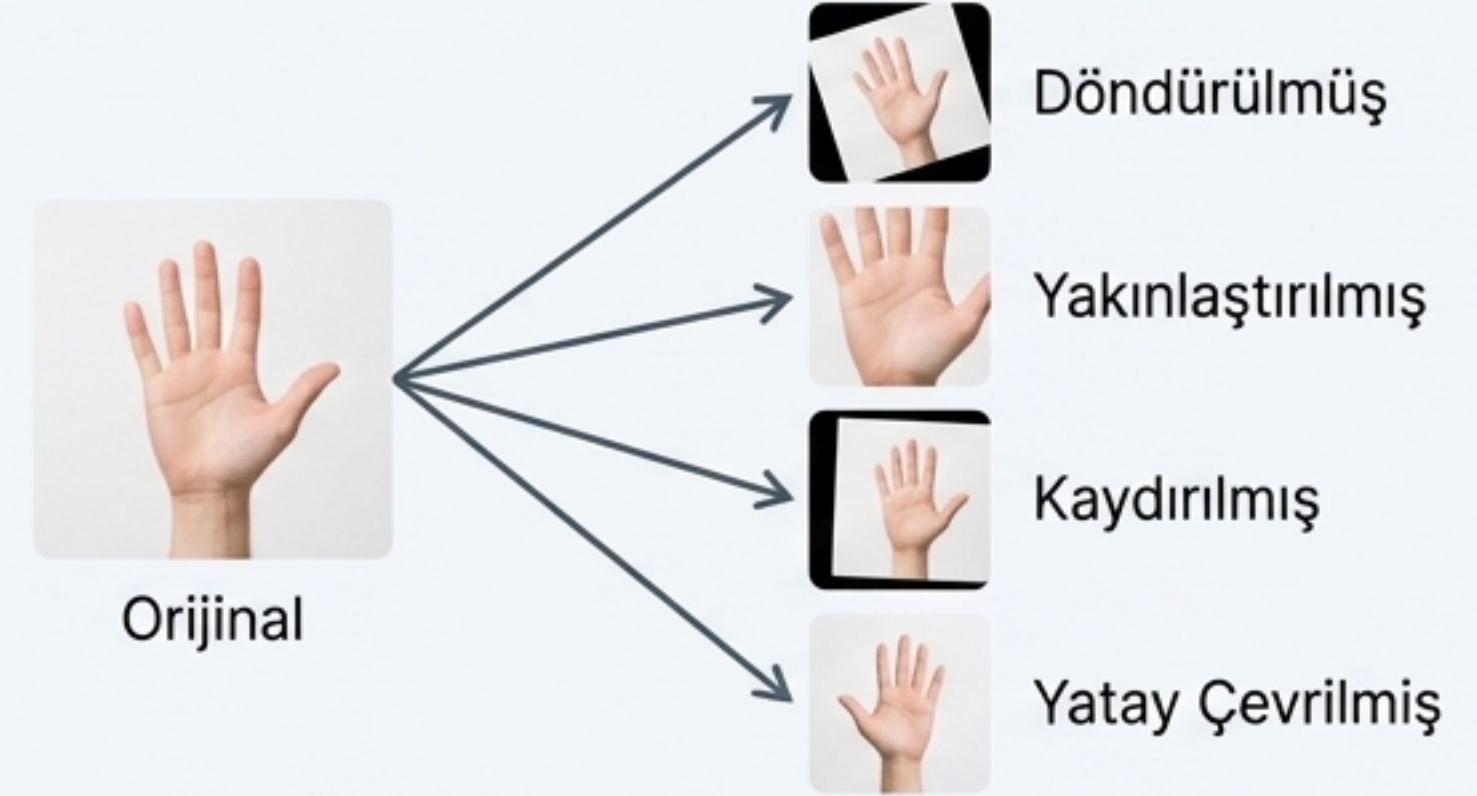
Modeli Gerçek Dünyaya Hazırlamak: Veri Artırma

Problem:

Orijinal veri setimiz kontrollü stüdyo koşullarında çekilmiş. Modelimiz farklı açılara, aydınlatmaya veya küçük el pozisyonu değişikliklerine karşı ne kadar dayanıklı?

Çözüm: `ImageDataGenerator` ile Veri Artırma

Mevcut eğitim görsellerimizden programatik olarak yeni, değiştirilmiş örnekler üretiyoruz. Bu, modelin hiç görmediği varyasyonları öğrenmesini sağlayarak genelleme yeteneğini kökten artırır.



```
train_datagen_augmented = ImageDataGenerator(  
    rescale=1./255,  
    rotation_range=20, ← Görüntüyü 20 dereceye  
                        kadar rastgele döndürür.  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.2,  
    zoom_range=0.2, ← %20'ye kadar yakınlaştırır.  
    horizontal_flip=True, ← Yatayda rastgele çevirir.  
    validation_split=0.2  
)
```

Geldiğimiz Nokta ve “Daha İyisi Mümkün mü?”

Şu Ana Kadar Neler Yaptık?

- ✓ Sıfırdan çalışan bir CNN modeli inşa ettik.
- ✓ **Aşırı Öğrenme** sorununu EarlyStopping ile kontrol altına aldık.
- ✓ **Dayanıklılık** eksikliğini Veri Artırma ile giderdik.

Yeni Soru:

Kendi inşa ettiğimiz model oldukça iyi. Ancak, Google gibi endüstri liderlerinin milyarlarca görüntü ile eğittiği devasa modellerin birikiminden faydalanabilir miyiz?

Cevap:

Evet. Transfer Learning ile.

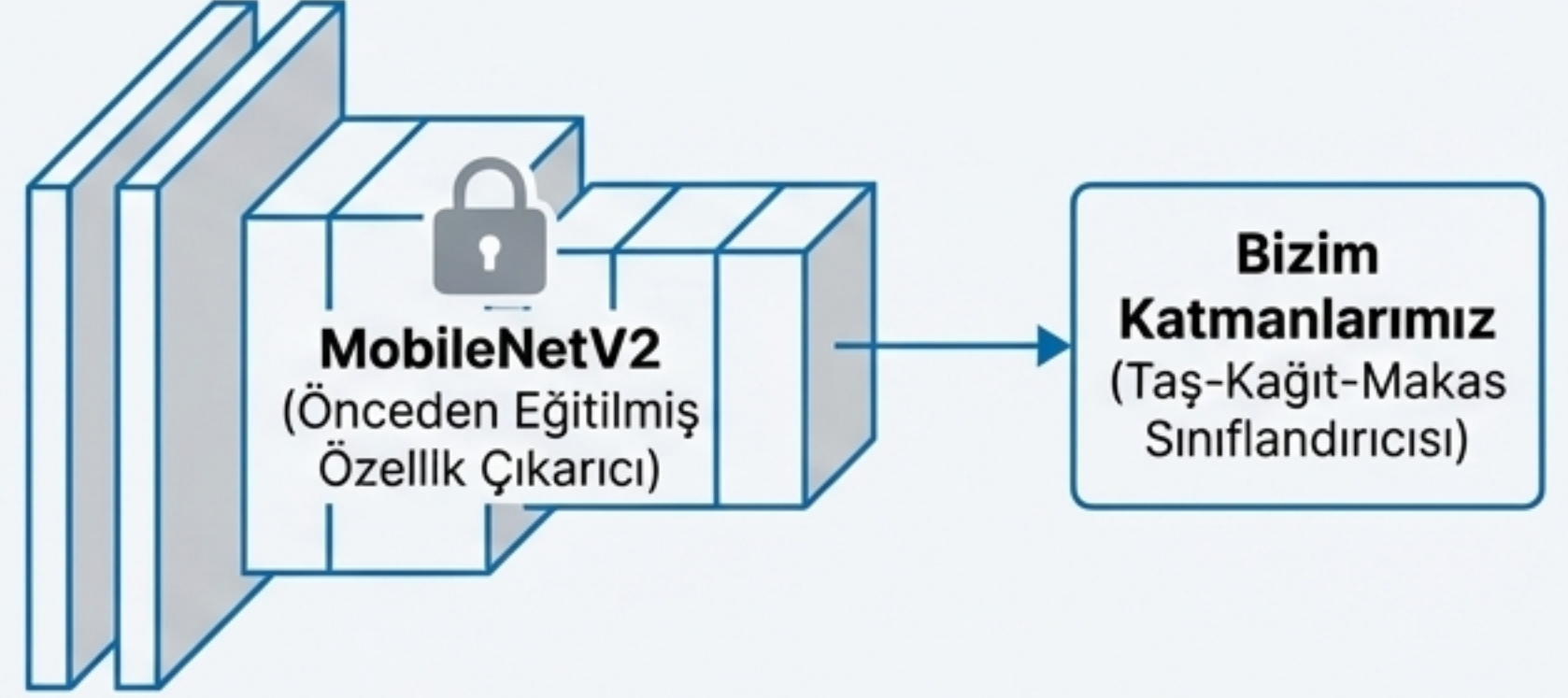
Devlerin Omuzlarında Yükselmek: MobileNetV2

Konsept: Transfer Learning

Sıfırdan 'görmeyi' öğrenmek yerine, Google tarafından ImageNet veri setinde eğitilmiş **MobileNetV2** modelinin önceden öğrenilmiş görüntü tanıma yeteneklerini kullanıyoruz.

Stratejimiz:

1. **Temel Modeli Yükle:** MobileNetV2'nin evrişimli katmanlarını yüklüyoruz, ancak son sınıflandırma katmanını ('include_top=False') hariç tutuyoruz.
2. **Dondur:** Başlangıçta bu temel modelin ağırlıklarının değişmesini engelliyoruz ('base_model.trainable = False'). Amacımız, mevcut bilgiyi korumak.
3. **Kendi Katmanlarımızı Ekle:** Dondurulmuş temelin üzerine, kendi Taş-Kağıt-Makas problemimize özel 'GlobalAveragePooling2D', 'Dropout' ve 'Dense' katmanları ekliyoruz.



```
base_model = MobileNetV2(input_shape=(150, 150, 3),
                          include_top=False,
                          weights='imagenet')
base_model.trainable = False
```

```
# Kendi katmanlarımızı ekliyoruz...
x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = Dropout(0.5)(x)
outputs = Dense(3, activation='softmax')(x)
```

Hassas Ayar Sanatı: İki Aşamalı Eğitim Stratejisi

Tek bir eğitim yerine, iki aşamalı, daha kontrollü bir yaklaşım benimsiyoruz:

Aşama 1: Özellik Çıkarımı (Feature Extraction)

Ne Yapılır? Sadece kendi eklediğimiz yeni sınıflandırma katmanları eğitilir. Temel model donuk kalır.

Amaç: Yeni katmanlarımızın MobileNetV2'nin ürettiği zengin özellikleri nasıl yorumlayacağını öğrenmesini sağlamak.

Parametreler: 10 epoch, daha yüksek öğrenme oranı (0.001).

Aşama 2: İnce Ayar (Fine-Tuning)

Ne Yapılır? Temel modelin son 40 katmanı 'çözülür' (base_model.trainable = True).

Amaç: Modelin tamamının, problemimize daha da özel hale gelmesi için küçük güncellemeler yapmasına izin vermek.

Parametreler: 20 epoch, **çok düşük** bir öğrenme oranı (0.00001) ile mevcut bilgiyi bozmamak için.

```
AŞAMA 1 { # AŞAMA 1: Özellik Çıkarımı Eğitimi Başlatılıyor...
            model_pro.compile(optimizer=Adam(learning_rate=0.001), ...)
            history_stage1 = model_pro.fit(...)

AŞAMA 2 { # AŞAMA 2: İnce Ayar (Fine-Tuning) Başlatılıyor...
            base_model.trainable = True
            # ...
            model_pro.compile(optimizer=Adam(learning_rate=0.00001), ...)
            history_stage2 = model_pro.fit(...)
```

Nihai Model: Dayanıklı, Hassas ve Güvenilir

Bu profesyonel yaklaşım, projemizin en yüksek performansını sağladı.

Gelişmiş Geri Çağrılar (Callbacks):

- **`ModelCheckpoint`**: Sadece en iyi doğrulama başarımına (``val_accuracy``) sahip modeli diske kaydeder.
- **`ReduceLROnPlateau`**: Modelin öğrenmesi 3 epoch boyunca yavaşladığında, öğrenme oranını otomatik olarak düşürerek sıkışıp kalmasını engeller.

Epoch 7/20

...

Epoch 8/20

`val_accuracy` improved from 0.9872 to 0.9905,
saving model to `rps_model_pro_best.keras`

...

Epoch 11/20

...

Epoch 12/20

`ReduceLROnPlateau` reducing learning rate to
`1.99999...e-06`.

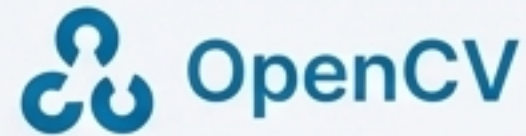
Sonuç:

Hibrit veri setimiz üzerinde %99'un üzerinde doğrulama başarımı ile projenin en güçlü modelini elde ettik:

`rps_model_pro_best.keras`

Koddan Gerçeğe: İnteraktif Taş-Kağıt-Makas Oyunu

Tüm bu emeğin somut çıktısı: Gerçek zamanlı çalışan bir oyun.



Bilgisayarın kamerasından anlık görüntü akışını sağlar.



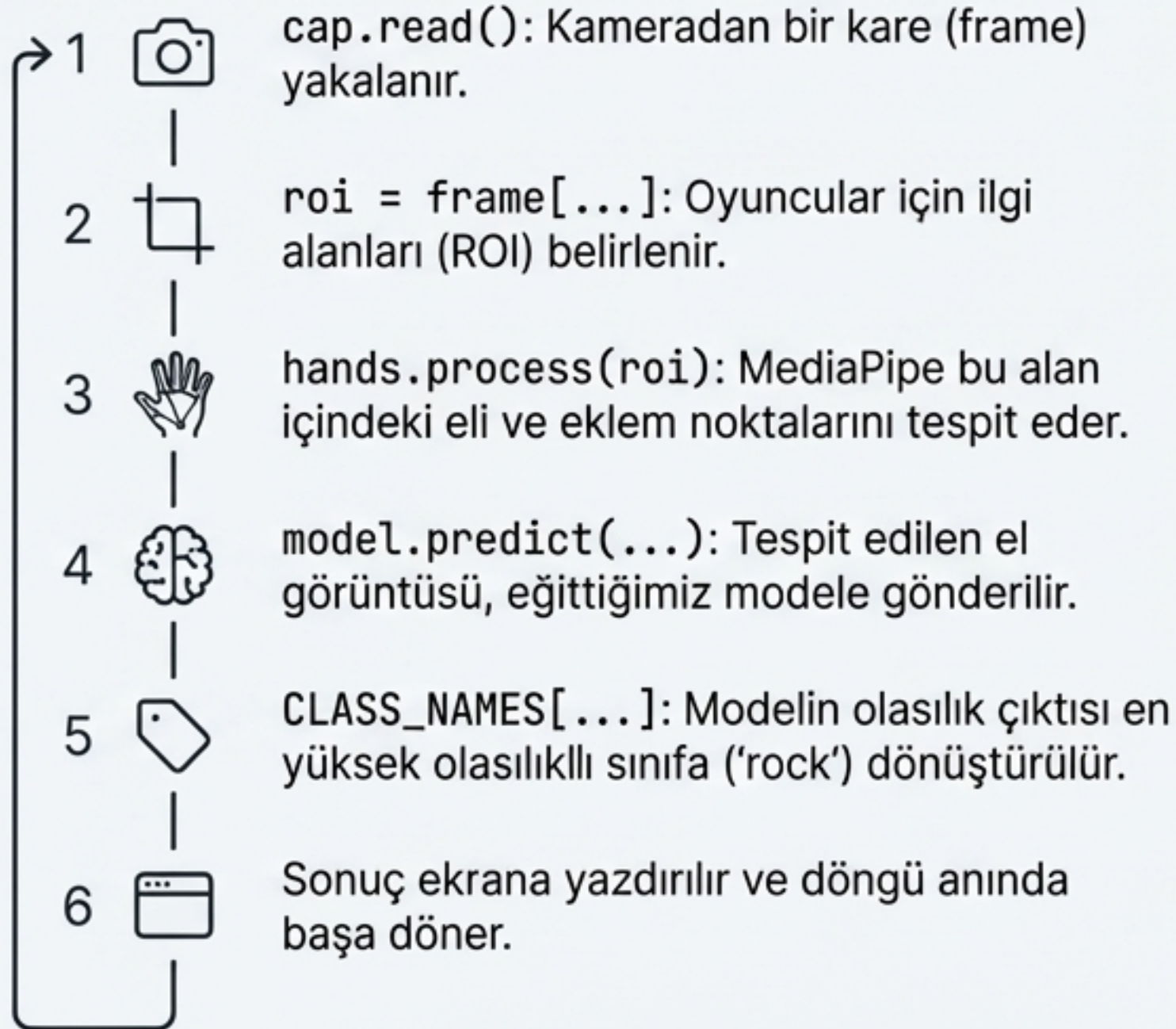
Görüntüdeki elleri ve parmak eklemlerini inanılmaz bir hassasiyetle tespit eder. Bu, modelimizin sadece doğru bölgeye odaklanmasını sağlar.



MediaPipe tarafından tespit edilen el bölgesini alır, Taş, Kağıt veya Makas olarak sınıflandırır.

Kamera, Tespit ve Tahmin Döngüsü

Oyunun her bir karesinde arka planda çalışanlar:



```
# --- Ana Oyun Döngüsü ---  
while True:  
    ret, frame = cap.read() # 1  
  
    roi1 = frame[100:400, 50:350] # 2  
    roi2 = frame[100:400, 340:640] # 2  
  
    processed_hand1, landmarks1 = process_hand(roi1, hands1) # 3  
  
    if processed_hand1 is not None:  
        prediction1 = model.predict(processed_hand1) # 4  
        move1 = CLASS_NAMES[np.argmax(prediction1)] # 5  
  
    # Sonuç ekrana yazdırılır (cv2.putText)... # 6
```

Serüvenimizden Çıkardığımız Dersler

Bu proje, bir fikri hayata geçirirken izlenen yolların bir özetidir. Inter Regular (#212529).

Ana Öğrenimlerimiz:



1. **Basit Başla, İteratif Geliştir:** Mükemmel modeli ilk denemede aramayın. Temel bir modelle başlayın, sonuçlarını analiz edin ve onu adım adım iyileştirin.



2. **Veriyi Görselleştir ve Dinle:** Eğitim grafikleri, modelinizin size ne anlatmaya çalıştığını gösteren en iyi araçtır. Overfitting gibi sorunları erken teşhis etmenizi sağlar.



3. **Veri, Kraldır:** İyi bir model kadar, çeşitli ve zengin bir veri seti de önemlidir. **Veri Artırma**, modelin genelleme gücünü artıran sihirli bir dokunuştur.



4. **Devlerin Omuzlarında Yüksel:** **Transfer Learning**, size haftalarca sürecektir eğitim sürecini kazandırabilecek inanılmaz güçlü bir tekniktir. Tekerleği yeniden icat etmeyin.

Teşekkürler!